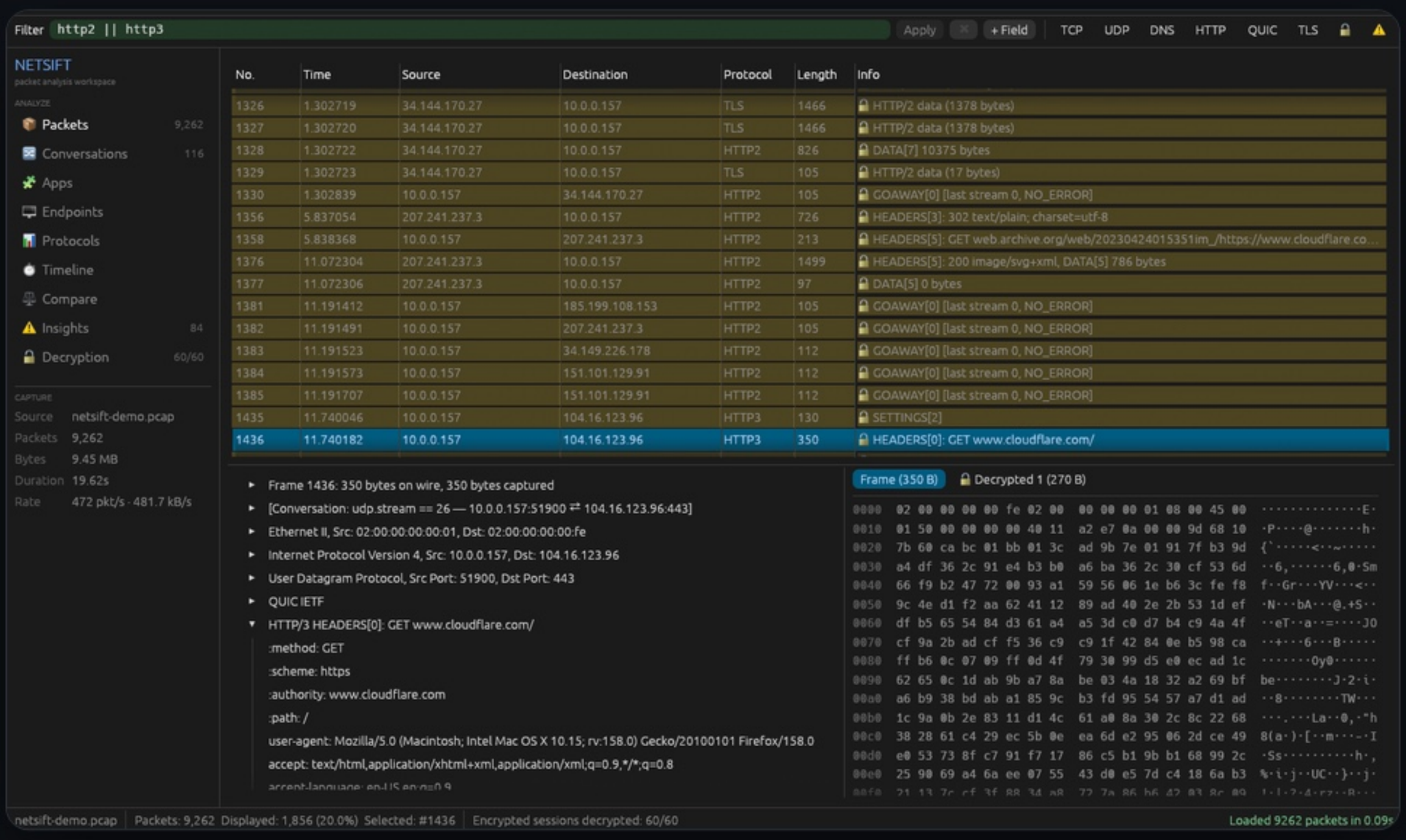
Side project · built in Rust

# I built a packet analyzer with the features Wireshark users keep asking for.

NetSift: live capture, pcap/pcapng, and live decryption of HTTPS, HTTP/2 and HTTP/3.




Swipe →

01

# Which app sent that packet?

Every connection is attributed to the local process that owns it — **during live capture, no root needed** for your own apps.

| App | Processes | Connections | Packets | ↑ Sent | ↓ Received |
|---|---|---|---|---|---|
| Firefox | firefox (92402), firefox (92412), fire… | 8 | 2,529 | 194.4 kB | 2.32 MB |
| git-remote-http | git-remote-http (92440) | 1 | 132 | 3.3 kB | 124.4 kB |
| curl | curl (92436), curl (92437) | 2 | 62 | 3.3 kB | 12.2 kB |

- Firefox, git and curl — each with its own bytes, connections and top hosts
- Filter with `process == "firefox"`, `app == "…"`, `pid == …`
- macOS kernel metadata (PKTAP) + socket table; Linux via /proc

> **Wireshark:** requested since bug 1184. It can show process info only if the capture file already contains it.

02

# HTTPS, HTTP/2 and HTTP/3 — decrypted live

One click launches Firefox or Chrome with key logging, loads the keys and starts capturing. Requests decode as you browse.




- TLS 1.2/1.3, QUIC v1/v2, HPACK and QPACK header decoding
- Handles key updates and Encrypted Client Hello (ECH)
- Real Firefox session: **60 / 60** encrypted sessions decrypted

03

# Every request as a waterfall

Connection setup, server wait and download for each HTTP/1, /2 and /3 request — like browser devtools, but built from packets.

| URL | Ver | Process | Size | Time |
|---|---|---|---|---|
| www.cloudflare.com/ | h3 | | 109.6 kB | 103.3 ms |
| www.cloudflare.com/fonts/Kunst%20Grotesk%2… | h3 | | 19.5 kB | 34.2 ms |
| www.cloudflare.com/fonts/Kunst%20Grotesk%2… | h3 | | 22.2 kB | 38.1 ms |
| www.cloudflare.com/static/hero-poster.avif | h3 | | 17.8 kB | 38.3 ms |
| firefox.settings.services.mozilla.com/v2/buckets/… | h2 | | 166 B | 93.8 ms |
| firefox.settings.services.mozilla.com/v2/buckets/… | h2 | | 179 B | 25.7 ms |
| firefox.settings.services.mozilla.com/v2/buckets/… | h2 | | 715 B | 25.2 ms |
| www.cloudflare.com/fonts/STKBureauSans-Book.… | h3 | | 8.1 kB | 37.4 ms |
| www.cloudflare.com/icons.svg | h3 | | 193.2 kB | 97.1 ms |
| www.cloudflare.com/fonts/STKBureauSans-Regul… | h3 | | 8.2 kB | 38.2 ms |
| www.cloudflare.com/_astro/_layout.astro_astro_… | h3 | | 3.5 kB | 39.9 ms |
| www.cloudflare.com/_astro/_separator.BfvVE_KL… | h3 | | 79.6 kB | 57.2 ms |
| www.cloudflare.com/_astro/index.Cm-titPL.css | h3 | | 1.2 kB | 38.0 ms |
| www.cloudflare.com/_astro/index.CN3UJUeJ.css | h3 | | 1.3 kB | 36.5 ms |
| www.cloudflare.com/_astro/DomainSearchApp.W… | h3 | | 1.9 kB | 45.1 ms |
| www.cloudflare.com/_b/v/latest/entry.js | h3 | | 1.2 kB | 276.0 ms |




- Drag the time axis to zoom; click a request to jump to its packets
- Export to **HAR** for devtools and HAR viewers

> **Wireshark:** no request waterfall, no HAR export.

04

# Diff two captures

Client side vs server side: what never arrived, what appeared, and how long each packet took between the two points.

| 9,262 | 9,218 | 44 | 18.3 ms |
|---|---|---|---|
| in capture A | in capture B | missing from B | B – A delay |

Missing from B (44)

```
#2200  104.16.123.96 → 10.0.0.157  DATA[20] 3669 bytes
#2201  104.16.123.96 → 10.0.0.157  1-RTT PKN=530 KP0: STREAM(72)
#2202  104.16.123.96 → 10.0.0.157  DATA[88] 672 bytes
#2203  104.16.123.96 → 10.0.0.157  1-RTT PKN=532 KP0: STREAM(168)
#2204  104.16.123.96 → 10.0.0.157  1-RTT PKN=533 KP0: STREAM(144)
#2205  104.16.123.96 → 10.0.0.157  1-RTT PKN=534 KP0: STREAM(120)
#2206  104.16.123.96 → 10.0.0.157  1-RTT PKN=535 KP0: STREAM(172)
#2207  104.16.123.96 → 10.0.0.157  DATA[196] 300 bytes, DATA[196] 0 bytes
```

- Matches on ports, TCP sequence numbers and payload
- Ignores TTL, IP ID, checksums, link headers and NAT
- `compare.missing` filters straight to the lost packets

> **Wireshark:** users have asked for a built-in capture diff for years; today it takes third-party tools.

05

# Search 1 million packets in milliseconds

1.03M packets · 1 GB capture built from real Firefox traffic

**8–73 ms**

per search once loaded

tshark re-reads the file: 4.1–5.3 s per search

**7.2–7.5 s**

load + decrypt HTTPS & HTTP/3

tshark: 11.0–11.7 s

Apple M4 Pro · tshark 4.6.8 · same filters, both printing matches.
Honest trade-offs: with no decryption, tshark's single pass is a bit faster (4.1–5.3 s vs 5.4–5.9 s), and NetSift uses ~3× the memory because it keeps everything loaded for instant search.

06

# Built around a workflow

Capture → filter → navigate → inspect → dig in. Follow any stream as readable HTTP messages:

```
→ HEADERS[0]: GET www.cloudflare.com/  (packet #1436)
    :method: GET
    :scheme: https
    :authority: www.cloudflare.com
    :path: /
    user-agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:158.0) Gecko/20100101 Firefox/158.0
    accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
    accept-language: en-US,en;q=0.9
    accept-encoding: gzip, deflate, br, zstd
    alt-used: www.cloudflare.com
    upgrade-insecure-requests: 1
    sec-fetch-dest: document
    sec-fetch-mode: navigate
    sec-fetch-site: none
    priority: u=0, i
```

```
$ netsift query big.pcap -f 'tls.sni contains "github"'
$ netsift http  capture.pcap -k keys.log
$ netsift diff  client.pcap server.pcap
```

Plus Insights (resets, retransmissions, DNS/HTTP errors), bookmarks and a headless CLI for scripting.

07

# Tested against real stacks, not mock data

✓ Real **rustls** handshakes: every TLS 1.2 / 1.3 cipher suite

✓ Real **h2** and **quinn + h3** traffic, incl. forced QUIC key updates

✓ RFC 7541 (HPACK) and RFC 9001 (QUIC) test vectors

✓ Live capture tests: a short-lived `curl` attributed 6 / 6 runs

✓ Fuzzing: thousands of mutated packets, zero crashes

✓ Cross-checked against tshark on a real Firefox capture

~11,600 lines of Rust · egui · libpcap · RustCrypto

What's next?

# What do you miss most in Wireshark?

Tell me in the comments — the most-requested feature gets built next.

- Process info saved into pcapng
- Lower memory for 10 GB+ captures
- Brotli/gzip bodies in HAR
- Windows support